Reviewer Pack — Spectral Gap Exponential Lower Bound on Tseitin Resolution L...

Entry cd7a96f56994 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 04:34:19 UTC

Spectral Gap Exponential Lower Bound on Tseitin Resolution Length

Entry ID: `cd7a96f56994` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 04:34:19 UTC

1. Conjecture

Field A (mathematical branch): Spectral Graph Theory **Field B** (complexity object): Resolution Proof Length

Statement:

> For a d-regular graph G with second eigenvalue λ_2, the Tseitin formula on G requires Resolution refutations of length $\geq 2^{\Omega(1/\lambda_2)}$. For graphs with $\lambda_2 \geq 1 - \varepsilon$, the refutation length is polynomial in n.

Rationale (proposer's reasoning):

> Expander graphs (small λ_2) force long Resolution proofs via cumulative entropy arguments. The spectral gap directly quantifies expansion properties, linking graph structure to proof complexity via the interplay between clause dependencies and eigenvalue decay.

Taxonomy category: `TSEITIN_RES` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `34804c340a5b7c30`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_d_regular_graph(n, d):
    if n * d % 2 != 0:
        raise ValueError("d must be even")

    G = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if (u, v) not in edges_added and (v, u) not in edges_added:
            G[u].append(v)
            G[v].append(u)
            edges_added.add((u, v))
            edges_added.add((v, u))

    for _ in range(n * d // 2):
        while True:
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            if len(G[u]) < d and len(G[v]) < d and (u, v) not in edges_added and (v, u) not in edg
                add_edge(u, v)
                break

    return G

def power_iteration(A, num_iterations=100):
    n = len(A)
    x = [random.random() for _ in range(n)]
    x = [x_i / sum(x) for x_i in x]

    for _ in range(num_iterations):
        x = [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
        x = [x_i / sum(x) for x_i in x]

    return max(x), min(x)

def tseitin_formula_length(G):
    n = len(G)
```

```python
    # Simplified estimation based on graph complexity
    return 2 ** (n * math.log2(n))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    d = 4  # Example degree, must be even
    n = 30  # Number of vertices

    G = generate_d_regular_graph(n, d)
    λ₂, _ = power_iteration(G)

    if λ₂ >= 1 - 1e-6:
        return {
            "metric_name": "Tseitin Formula Length",
            "metric_value": tseitin_formula_length(G),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Graph with λ₂ ≥ 1 - ε"
        }

    length = tseitin_formula_length(G)
    c = 0.1
    if length >= 2 ** (c / λ₂):
        return {
            "metric_name": "Tseitin Formula Length",
            "metric_value": length,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "Tseitin Formula Length",
            "metric_value": length,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Graph with λ₂ = {λ₂}, length < 2^{c/λ₂}"
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 10000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    std_length = math.sqrt(sum((r["metric_value"] - mean_length) ** 2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fr
    elif support_fraction >= 0.8:
```

```
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fr
    else:
        counterexample = min((r["counterexample"] for r in results if not r["conjecture_holds"]),
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'Tseitin Formula Length', 'metric_value': 32.0, 'instances_tested': 1, 'conjec

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6219b7f4.py", line 79, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6219b7f4.py", line 60, in run_trial
    G = generate_d_regular_graph(n, d)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6219b7f4.py", line 31, in generate_d_re
    raise ValueError("d must be even")
ValueError: d must be even
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to invalid d parameter (must be even), preventing valid data collection. Pre-registered support condition (80% seeds) cannot be evaluated. | next: Fix test parameters to use even d values for d-regular graph generation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111249
2	propose	ollama_remote	qwen3:8b	0	0	56170
3	preregistration	ollama_remote	qwen3:8b	0	0	24458
4	novelty	ollama_remote	qwen3:8b	0	0	20874
5	novelty	ollama_remote	qwen3:8b	0	0	16060
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12689
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11093
8	judge	ollama_remote	qwen3:8b	0	0	16484

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 269077 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/cd7a96f56994.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd7a96f56994.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cd7a96f56994.tar.gz` (if generated)